Cyprus University of Technology

CSE 226
Algorithms and Complexity

Lecture 11

Elementary Graph Algorithms (Part 2)

Στοιχειώδεις Αλγόριθμοι Γραφημάτων (Μέρος 2)

Herodotos Herodotou

Contains edited slides from T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, D. Luebke, and M. Christodoulakis

1

Graph Searching

- Given: a graph (γράφημα) $G = (V, E)$, directed (κατευθυνόμενο) or undirected (μη κατευθυνόμενο)
- Goal: methodically explore every vertex (κορυφή) and every edge (ακμή)
- Ultimately: build a tree (δένδρο) on the graph
- **Idea**
  - Pick a vertex as the root (ρίζα)
  - Choose certain edges to produce a tree (δένδρο)
  - Note: might also build a forest (δάσος)

2

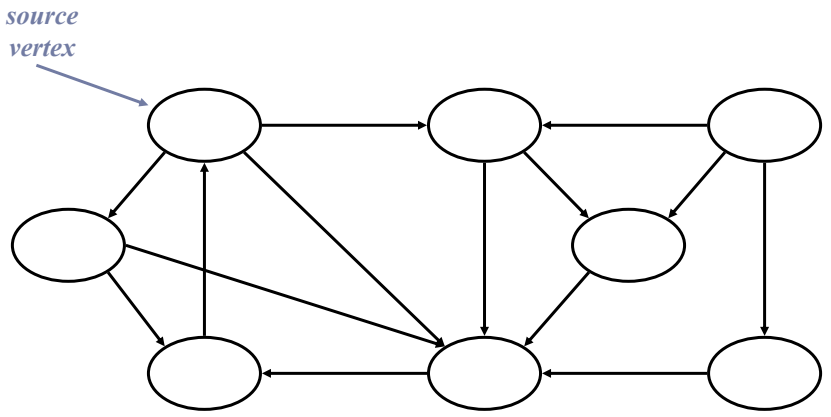
Depth-First Search

- **Depth-first search** (**Καθοδική Διερεύνηση**) is another strategy for exploring a graph
  - Explore "deeper" in the graph whenever possible
  - Edges are explored out of the most recently discovered vertex v that still has unexplored edges
  - When all of v's edges have been explored, backtrack to the vertex from which v was discovered

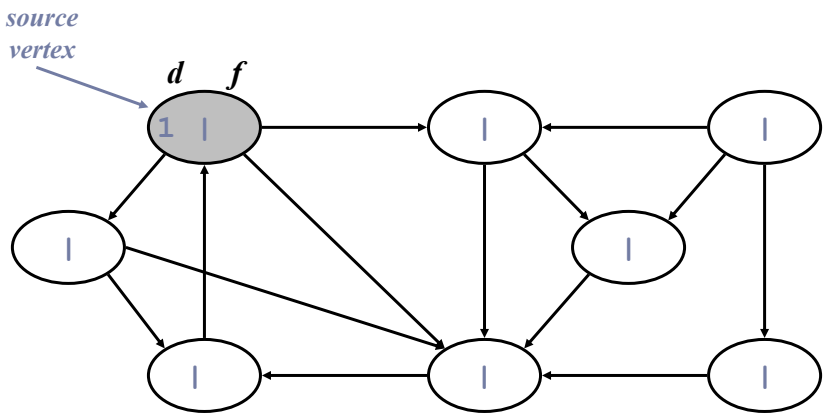
Depth-First Search

- Associate vertex "colors" to guide the algorithm
  - **white** = undiscovered (all vertices start white)
  - **gray** = discovered, but not finished (not done exploring from it)
  - **black** = finished (have found everything reachable from it)
- Record **timestamps** (**χρονοσφραγίδες**) for each vertex v
  - **Discovery time** (**χρόνος εντοπισμού**) d: when v is first discovered in the search (and grayed)
  - **Finishing time** (**χρόνος περάτωσης**) f: when v's adjacency list has been examined completely (and v is blackened)
  - Unique integers from 1 to $2|V|$
  - $d[v] < f[v]$

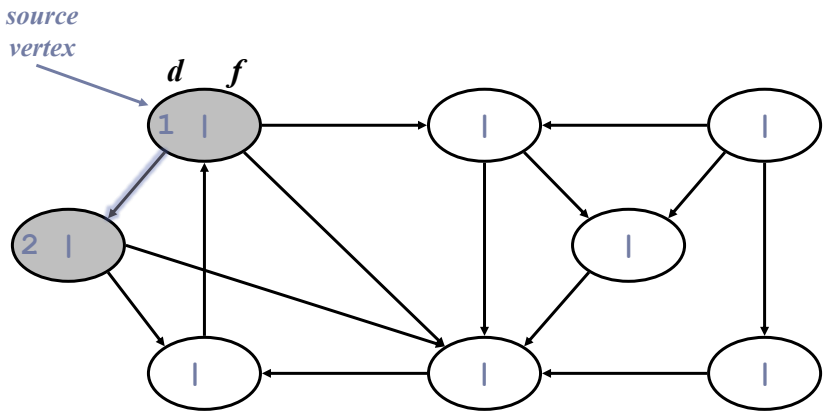
Depth-First Search: Example



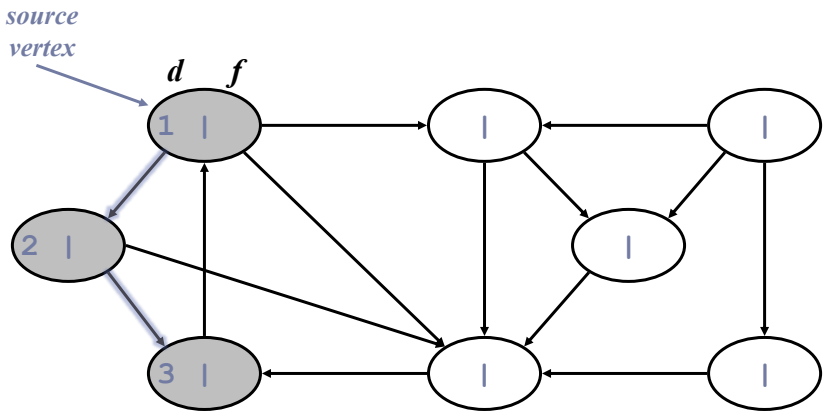
Depth-First Search: Example



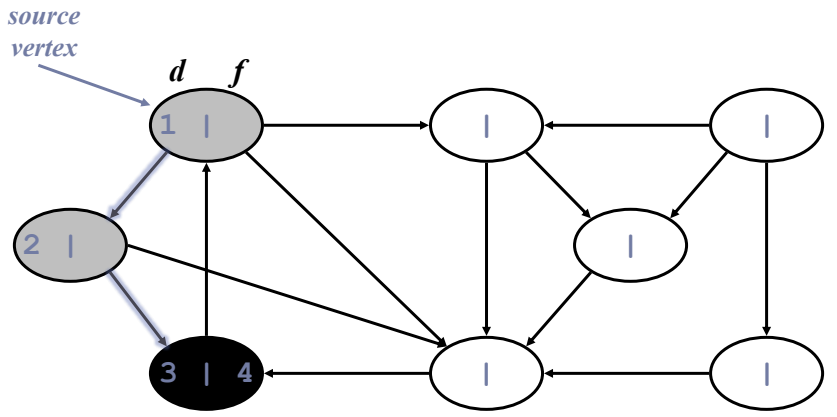
Depth-First Search: Example



Depth-First Search: Example

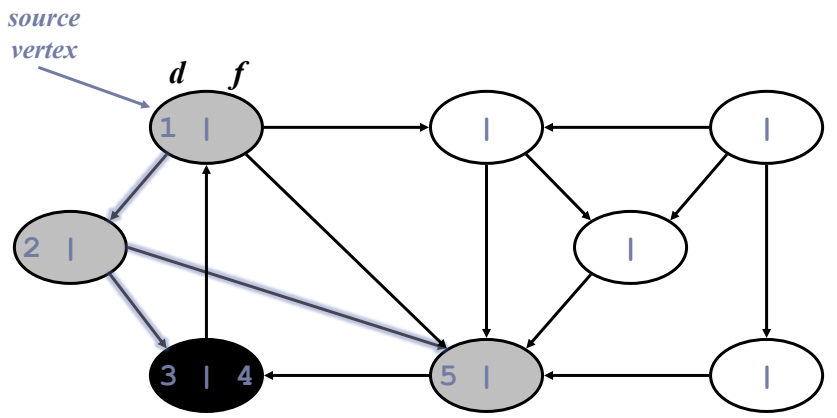


Depth-First Search: Example



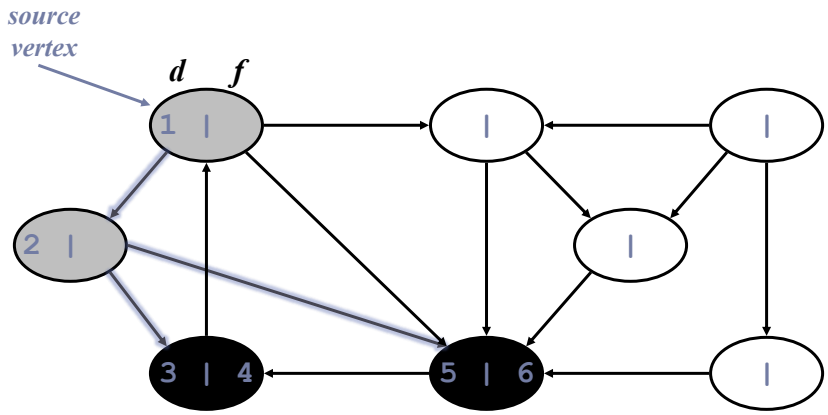
9

Depth-First Search: Example

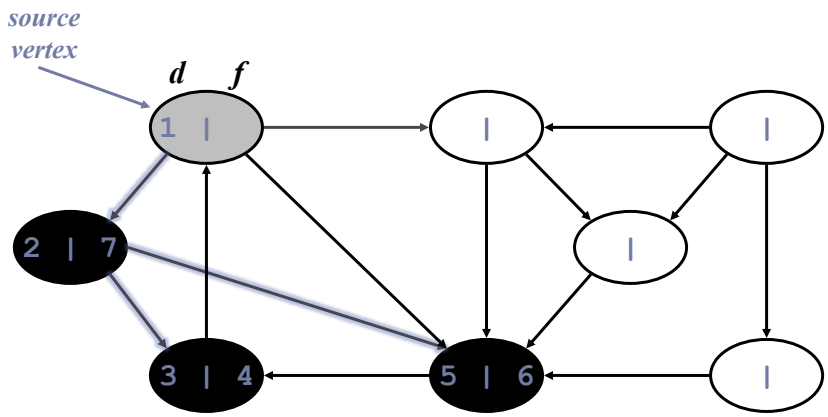


10

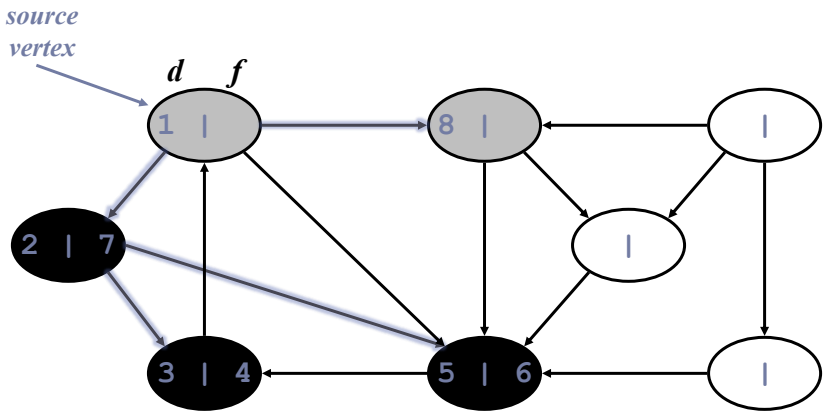
Depth-First Search: Example



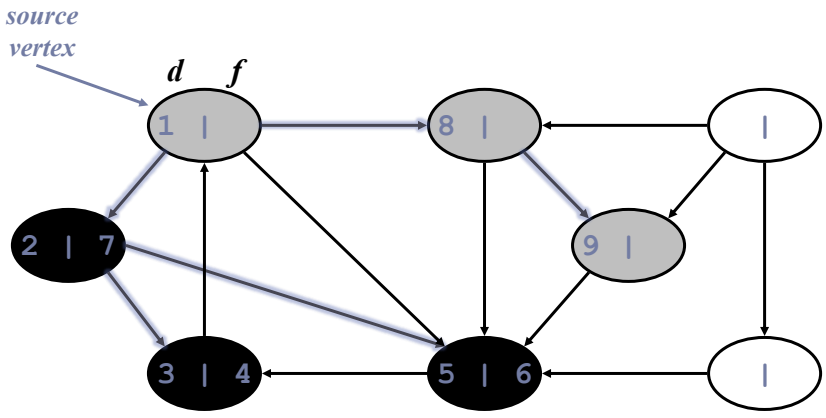
Depth-First Search: Example



Depth-First Search: Example

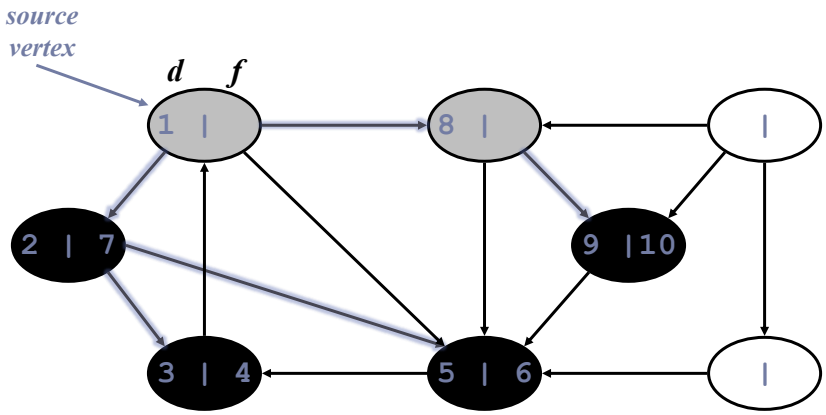


Depth-First Search: Example



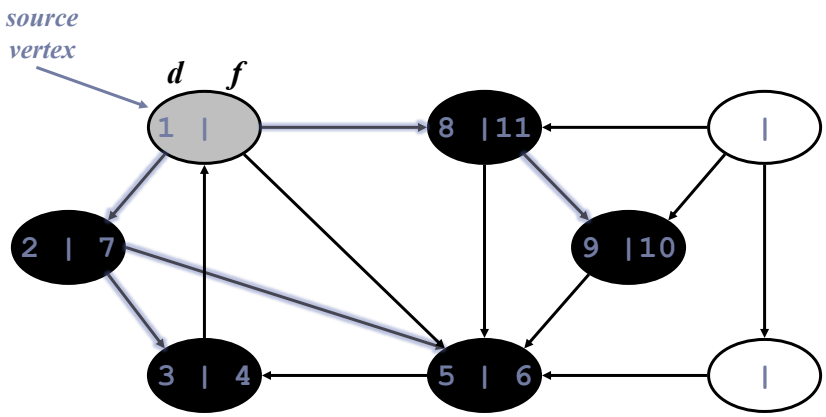
What is the structure of the grey vertices?
What do they represent?

Depth-First Search: Example



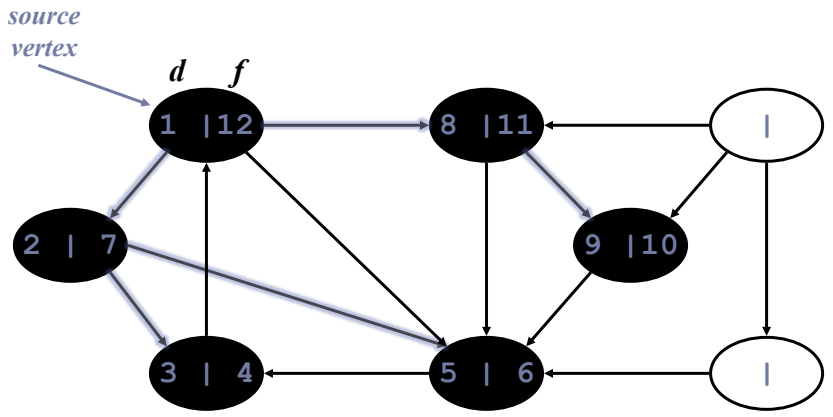
15

Depth-First Search: Example



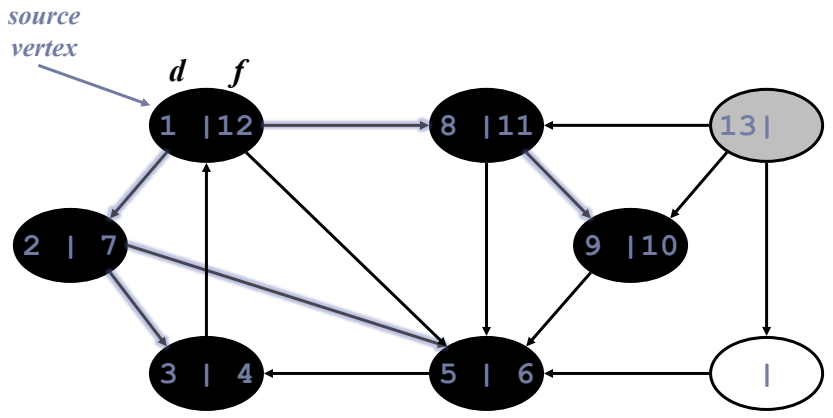
16

Depth-First Search: Example



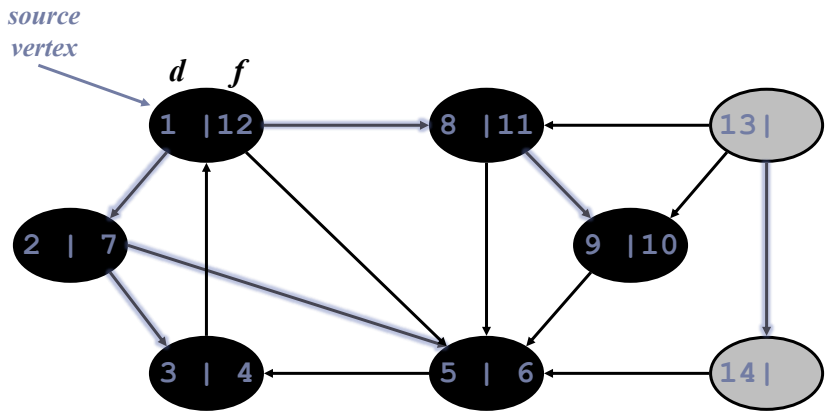
17

Depth-First Search: Example

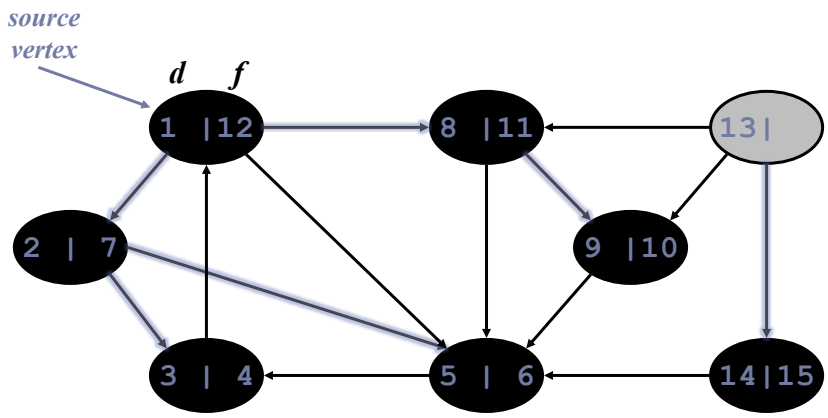


18

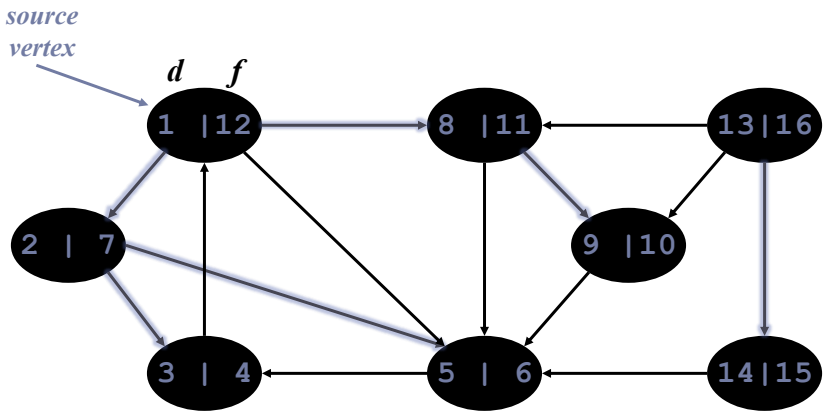
Depth-First Search: Example



Depth-First Search: Example



Depth-First Search: Example



DFS forms a **depth-first forest** (**καθοδικό δάσος**) with >=1 **depth-first trees** (**καθοδικά δένδρα**)

Depth-First Search: The Code

```
DFS(G) {
  for each vertex u ∈ G.V {
    u.color = WHITE;
    u.p = NULL;
  }
  time = 0;
  for each vertex u ∈ G.V {
    if (u.color == WHITE) {
      DFS_Visit(G, u);
    }
  }
}
```

```
DFS_Visit(G, u) {
  time = time + 1;
  u.d = time;
  u.color = GREY;
  for each v ∈ G.Adj[u] {
    if (v.color == WHITE) {
      v.p = u;
      DFS_Visit(G, v);
    }
  }
  time = time + 1;
  u.f = time;
  u.color = BLACK;
}
```

Depth-First Search: The Code

```
DFS(G) {
  for each vertex u ∈ G.V {
    u.color = WHITE;
    u.p = NULL;
  }
  time = 0;
  for each vertex u ∈ G.V {
    if (u.color == WHITE) {
      DFS_Visit(G, u);
    }
  }
}
```

```
DFS_Visit(G, u) {
  time = time + 1;
  u.d = time;
  u.color = GREY;
  for each v ∈ G.Adj[u] {
    if (v.color == WHITE) {
      v.p = u;
      DFS_Visit(G, v);
    }
  }
  time = time + 1;
  u.f = time;
  u.color = BLACK;
}
```

Will all vertices eventually be colored black? Yes

Depth-First Search: The Code

```
DFS(G) {
  for each vertex u ∈ G.V {
    u.color = WHITE;
    u.p = NULL;
  }
  time = 0;
  for each vertex u ∈ G.V {
    if (u.color == WHITE) {
      DFS_Visit(G, u);
    }
  }
}
```

```
DFS_Visit(G, u) {
  time = time + 1;
  u.d = time;
  u.color = GREY;
  for each v ∈ G.Adj[u] {
    if (v.color == WHITE) {
      v.p = u;
      DFS_Visit(G, v);
    }
  }
  time = time + 1;
  u.f = time;
  u.color = BLACK;
}
```

What is the running time? $\Theta(V + E)$

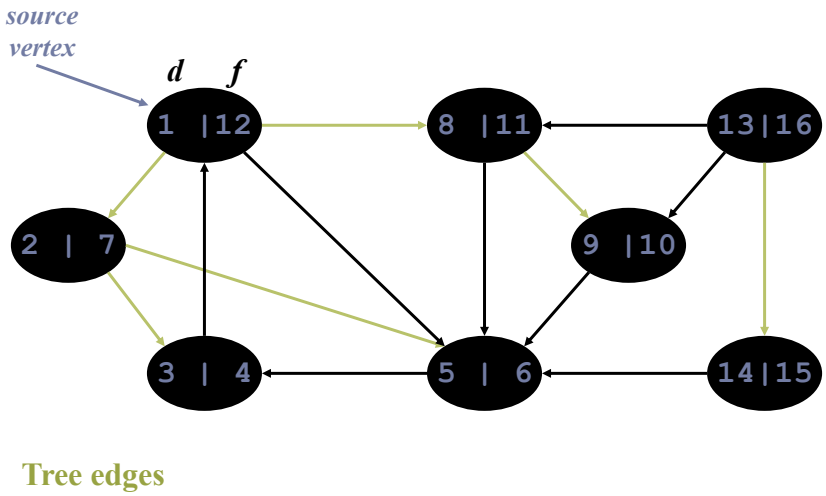
Depth-First Search: Running Time Analysis

- This running time argument is an informal example of **amortized analysis** (**αντισταθμιστική ανάλυση**)
  - Each loop in DFS_Visit can be attributed to an edge in the graph
  - Runs once per edge if directed graph, twice if undirected
  - Thus, loop will run in $O(E)$ time, algorithm $O(V + E)$
    - Considered "linear" for graph, because adjacency list requires $O(V + E)$ storage
- Important to be comfortable with this kind of reasoning and analysis

DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - **Tree edge** (**Δενδρική ακμή**): edge in the depth-first forest
    - When search encounters new (white) vertex
    - The tree edges form a **spanning forest** (**συνεκτικό δάσος**)
    - *Can tree edges form cycles? Why or why not?*

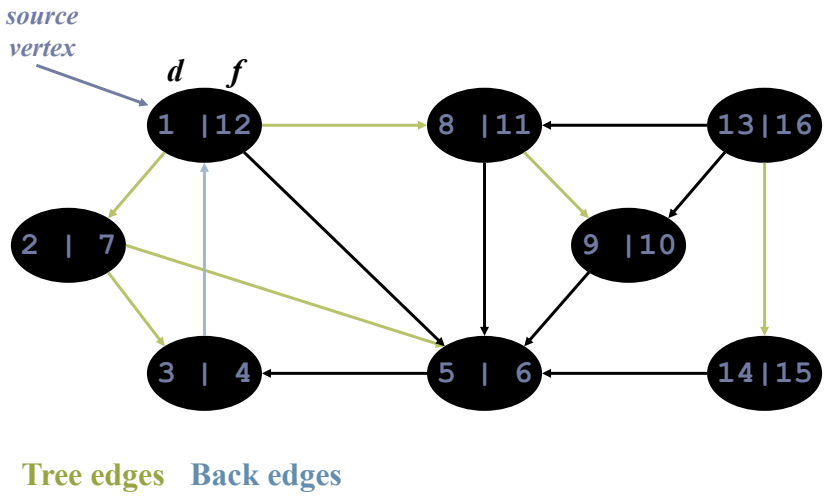
DFS: Example



DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - **Tree edge** (**Δενδρική ακμή**): edge in the depth-first forest
  - **Back edge** (**Ανιούσα ακμή**): connects a vertex to its ancestor (πρόγονο)
    - When search encounters a grey vertex (grey to grey)

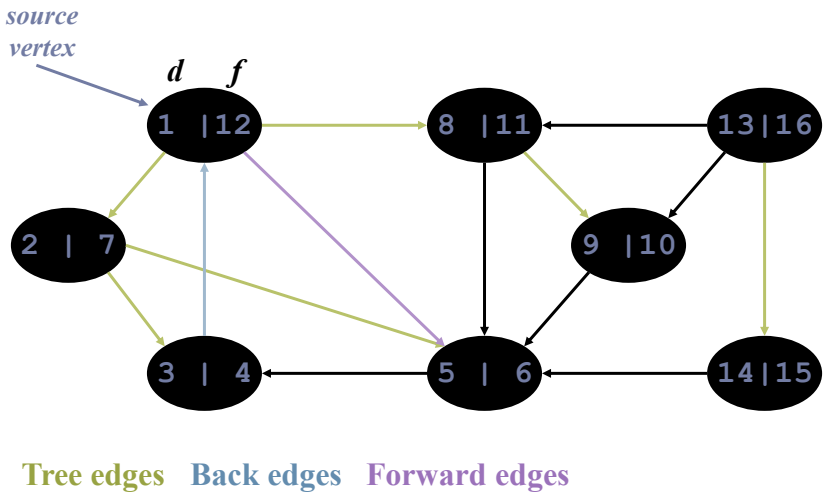
DFS: Example



DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - **Tree edge** (**Δενδρική ακμή**): edge in the depth-first forest
  - **Back edge** (**Ανιούσα ακμή**): connects a vertex to its ancestor
  - **Forward edge** (**Κατιούσα ακμή**): connects a vertex to its descendent (απόγονο)
    - Not a tree edge, though
    - From grey node to black node

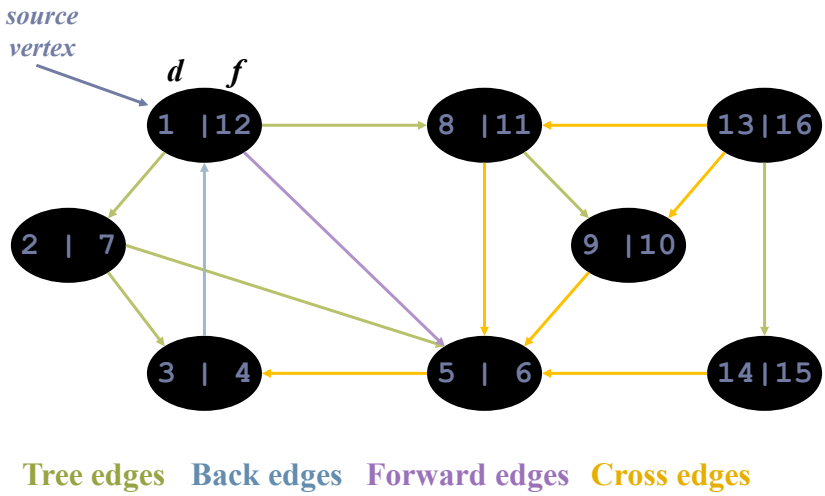
DFS: Example



DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - **Tree edge** (**Δενδρική ακμή**): edge in the depth-first forest
  - **Back edge** (**Ανιούσα ακμή**): connects a vertex to its ancestor
  - **Forward edge** (**Κατιούσα ακμή**): connects a vertex to its descendent
  - **Cross edge** (**Εγκάρσια ακμή**): between a tree or subtrees
    - From a grey node to a black node

DFS: Example



DFS: Kinds of edges

- DFS introduces an important distinction among edges in the original graph:
  - **Tree edge** (**Δενδρική ακμή**): edge in the depth-first forest
  - **Back edge** (**Ανιούσα ακμή**): connects a vertex to its ancestor
  - **Forward edge** (**Κατιούσα ακμή**): connects a vertex to its descendent
  - **Cross edge** (**Εγκάρσια ακμή**): between a tree or subtrees
- Note: tree & back edges are important; most algorithms don't distinguish forward & cross

DFS: Properties

- **Theorem (Parenthesis Theorem)**

  For all u, v, exactly one of the following holds:

  1. $d[u] < f[u] < d[v] < f[v]$ or $d[v] < f[v] < d[u] < f[u]$ and neither of u and v is a descendant of the other
  2. $\mathrm{d[u]} < \mathrm{d[v]} < \mathrm{f[v]} < \mathrm{f[u]}$ and v is a descendant of u
  3. $d[v] < d[u] < f[u] < f[v]$ and u is a descendant of v

- So $d[u] < d[v] < f[u] < f[v]$ cannot happen
- Like parentheses:
  - OK: ( ) [ ] ([ ]) [( )]
  - Not OK: ([ )] [( ])

DFS: Properties

- **Corollary**

  v is a descendant of u if and only if $d[u] < d[v] < f[v] < f[u]$

- **Theorem**

  In DFS of an *undirected* graph, we get only tree and back edges. No forward or cross edges

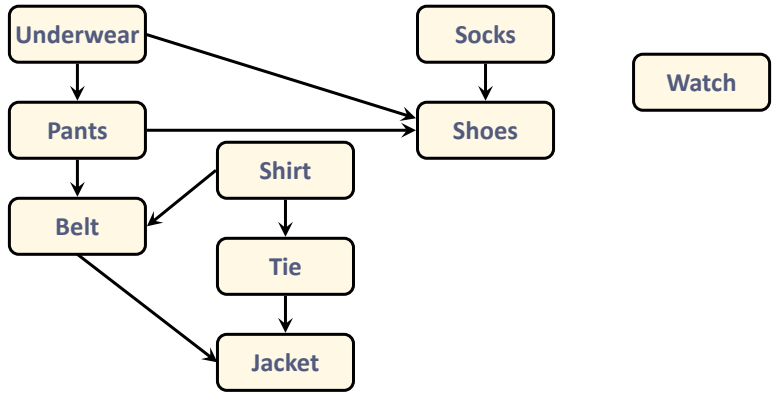
DFS And Graph Cycles

- **Lemma:** An undirected graph is **acyclic** (**άκυκλο**) if and only if a DFS yields no back edges
  - If acyclic, no back edges
    - Because a back edge implies a cycle
  - If no back edges, acyclic
    - No back edges implies only tree edges
    - Only tree edges implies we have a tree or a forest
    - Which by definition is acyclic
- Thus, can run DFS to find whether a graph has a cycle

Topological Sort

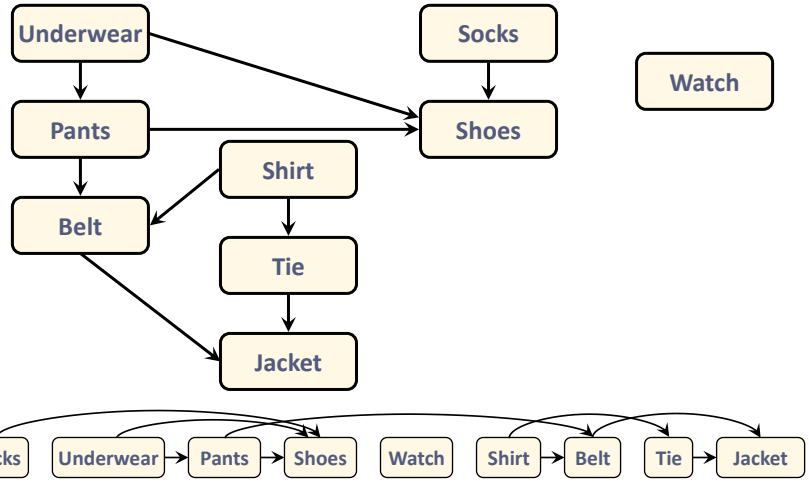
- **Topological sort** (**Τοπολογική ταξινόμηση**) of a Directed Acyclic Graph (DAG):
  - Linear ordering of all vertices in graph G such that vertex u comes before vertex v if edge $(u, v) \in G$
- Real-world example: getting dressed

Getting Dressed



39

Getting Dressed



40

Topological Sort Algorithm

```
Topological-Sort(G)
{
  Run DFS(G)
  When a vertex is finished, output it
  Vertices are output in reverse topological
    order
}
```

- Time: $\Theta(V + E)$
- Vertices are in order of *decreasing* finish times

Getting Dressed

